



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт кибербезопасности и цифровых технологий
Кафедра КБ-3 «Разработка программных решений и системное
программирование»**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
по дисциплине
«Сборка, тестирование и верификация программного продукта»**

Выполнил:

Студент группы БСБО-08-23

Птицын Фёдор Андреевич

Проверил:

К. Т. Н., доцент

Ивакин Сергей Николаевич

Москва, 2026 г.

Ход работы

В данной практической работе рассматривается решение задачи «Сдвоенная функция» с номером варианта 3.2.

Для варианта 3.2 используются следующие математические функции:

- левая функция $f(x) = \exp(x)$ – определена для всех действительных значений x ;
- правая функция $g(x) = 1 / (x-7)$ – имеет разрыв в точке $x=7$ (значение не определено, функция возвращает None).

Общая функция выбирает ветвь в зависимости от граничного значения n :

$$y = \begin{cases} \exp(x), & \text{при } x < n \\ 1/(x - 7), & \text{при } x \geq n \end{cases}$$

Программное решение состоит из двух файлов: `main.py` (Приложение А.1), содержащий основной код и описание функций, и `graph.py` (Приложение А.2), реализующий построение графика функции.

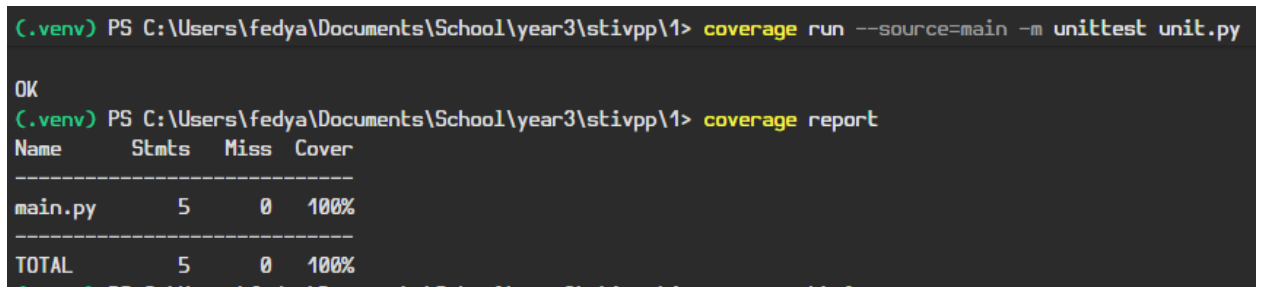
Для тестирования было использовано два метода: доктесты, тесты которого описаны в `main` и вызываются с помощью командной строки, и аналогичные юниттесты, реализованные в файле `unit.py` (Приложение А.3).

Написанные тесты осуществляют проверку различных сценариев:

1. для некорректных данных: передача строк, списков, сетов вместо чисел (функция не проверяет типы, поэтому должны возникать исключения);
2. для левой ветки: $x < n$ (убеждаемся, что выбирается правильная ветка и $\exp(x)$ вычисляется верно);
3. для правой ветки: $x \geq n$ (проверка выбора правой ветки и корректности $1 / (x - 7)$);
4. для точки разрыва: $x = 7$ ($g(x)$ не определена в 7, возникает исключение деления на 0; тест проверяет, что это обрабатывается корректно;

5. для множества значений: комбинации (x, n) в цикле с `subTest` (гарантирует, что все варианты работают одинаково, и упрощает локализацию ошибок).

Далее была проведена оценка покрытия кода тестами (`unittest`). Для этого в терминале была использована команда «`coverage run --source=main -m unittest unit.py`» (рисунок 1).

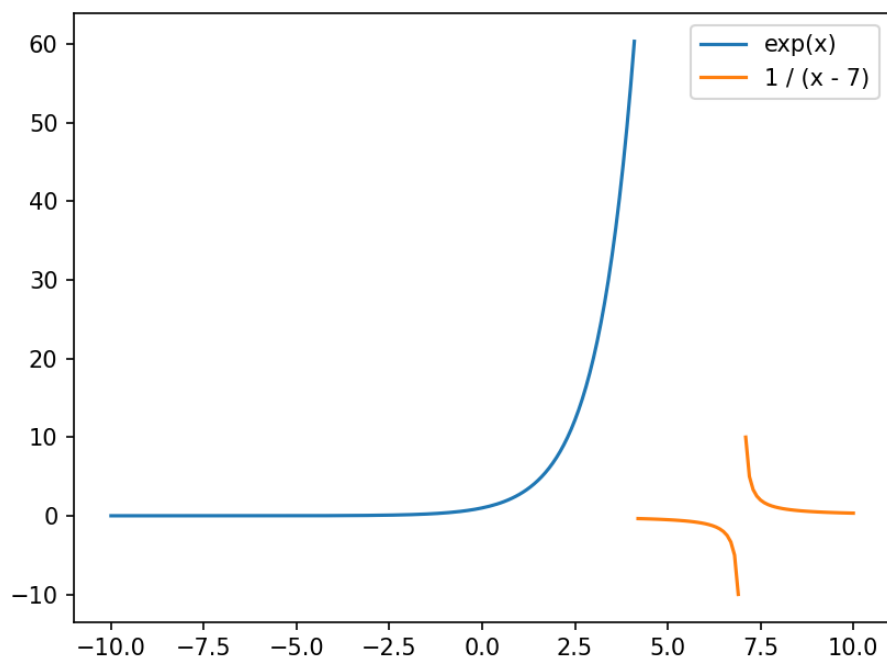


```
(.venv) PS C:\Users\fedya\Documents\School\year3\stivpp\1> coverage run --source=main -m unittest unit.py
OK
(.venv) PS C:\Users\fedya\Documents\School\year3\stivpp\1> coverage report
Name      Stmts  Miss  Cover
-----
main.py      5      0  100%
TOTAL        5      0  100%
```

Таким образом, основной функционал (расчёт сдвоенной функции) протестирован полностью.

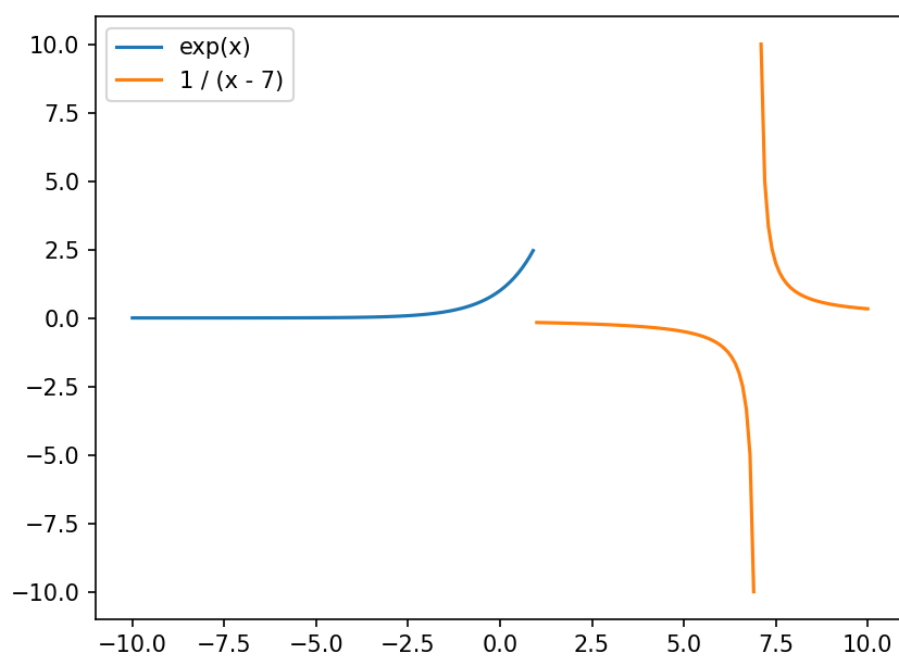
Заключительным этапом работы было тестирование визуализации. Функция `plot` строит общий график, где левая часть ($x < n$) отображается синим, а правая ($x \geq n$) – оранжевым. Функция была протестирована для двух разных значений n : 42 и 10.

На рисунке 2 видно, что функция терпит разрыв в двух местах: при $x = 42$ (смена функции для подсчета значения) и при $x = 7$ (так как используется правая ветка с возвращаемым значением `None` в точке разрыва).



$n = 42$

На рисунке 3 (график для $n = 10$) виден один разрыв – между левой и правой функцией.



$n = 10$

Приложения

Приложение А.1. Код файла main.py

```
# 3 - 2
# math.exp(x) -- 1 / (x - 7)
import math

def comp(x, n):
    '''

    >>> comp("2", "3")
    Traceback (most recent call last):
        ...
    TypeError: must be real number, not str

    >>> comp([2], 3)
    Traceback (most recent call last):
        ...
    TypeError: '<' not supported between instances of 'list' and 'int'

    >>> comp((2, ), 3)
    Traceback (most recent call last):
        ...
    TypeError: '<' not supported between instances of 'tuple' and 'int'

    >>> comp(2, dict(3))
    Traceback (most recent call last):
        ...
    TypeError: 'int' object is not iterable

    >>> comp({2}, 3)
    Traceback (most recent call last):
        ...
    TypeError: '<' not supported between instances of 'set' and 'int'

    >>> round(comp(4, 5), 1)
    54.6

    >>> comp(0, 2)
    1.0

    >>> round(comp(-2, 35), 2)
    0.14

    >>> round(comp(10, 5), 1)
    0.3
```

```

>>> round(comp(-7, -10), 2)
-0.07

>>> comp(7, 2)
Traceback (most recent call last):
...
ZeroDivisionError: division by zero

>>> test_pairs = [(1, 5), (2, 7), (6, 6), (8, 5), (7, 10)]
>>> results = [round(comp(x, n), 2) for x, n in test_pairs]
>>> results
[2.72, 7.39, -1.0, 1.0, 1096.63]

'''
if x < n:
    return math.exp(x)
return 1 / (x - 7)

```

Приложение A.2. Код файла graph.py

```

import math
from matplotlib import pyplot as plt

def f(x):
    return math.exp(x)

def g(x):
    if x != 7:
        return 1 / (x - 7)
    return None

def plot(n):
    fx = [i/10 for i in range(-100, n)]
    fy = [f(x) for x in fx]

    gx = [i/10 for i in range(n, 101)]
    gy = [g(x) for x in gx]

    plt.plot(fx, fy, label='exp(x)')
    plt.plot(gx, gy, label='1 / (x - 7)')
    plt.legend()
    plt.show()

if __name__ == '__main__':
    x = int(input("x: "))

```

```
n = int(input("n: " ))
plot(n)
```

Приложение А.3. Код файла unit.py

```
import unittest
import math
from main import comp

class TestComp(unittest.TestCase):

    def test_invalid_input(self):
        with self.assertRaises(TypeError):
            comp("2", "3")
        with self.assertRaises(TypeError):
            comp([2], 3)
        with self.assertRaises(TypeError):
            comp((2, ), 3)
        with self.assertRaises(TypeError):
            comp(2, set(3))
        with self.assertRaises(TypeError):
            comp({2}, 3)

    def test_exp_pos(self):
        res = comp(4, 5)
        self.assertEqual(res, math.exp(4))

    def test_exp_zero(self):
        res = comp(0, 2)
        self.assertEqual(res, 1)

    def test_exp_neg(self):
        res = comp(-2, 35)
        self.assertEqual(res, math.exp(-2))

    def test_hyper_pos(self):
        res = comp(10, 5)
        self.assertEqual(res, 1 / 3)

    def test_hyper_neg(self):
        res = comp(-7, -10)
        self.assertEqual(res, 1 / -14)

    def test_hyper_error(self):
```

```
        with self.assertRaises(ZeroDivisionError):
            comp(7, 2)

def test_multiple(self):
    test_cases = [
        (1, 5, math.e),
        (2, 7, math.exp(2)),
        (6, 6, -1),
        (8, 5, 1),
        (7, 10, 1 / math.exp(7)),
    ]
    for x, n, expected in test_cases:
        with self.subTest(x=x, n=n):
            self.assertAlmostEqual(comp(x, n), expected)

if __name__ == "__main__":
    unittest.main()
```